

Introduction à la Programmation

Qu'est-ce que la programmation ?

La programmation est le processus de création d'instructions pour qu'un ordinateur puisse exécuter des tâches spécifiques. Ces instructions sont écrites dans un langage de programmation, qui peut être compris par l'ordinateur. Certains langages de programmation populaires incluent Python, Java, C++, et JavaScript. Pour que le langage soit efficace, il doit être structuré et suivre des règles syntaxiques précises.

Compilé ou interprété ?

Les langages de programmation peuvent être classés en deux catégories principales : compilés et interprétés.

- **Langages compilés** : Le code source est transformé en code machine par un compilateur avant d'être exécuté. Cela signifie que le programme est converti en un format exécutable qui peut être directement exécuté par l'ordinateur. Exemples : C, C++. Par exemple, un programme écrit en C est compilé en un fichier exécutable qui peut être lancé sur le système d'exploitation (par exemple, un fichier `.exe` sur Windows).
- **Langages interprétés** : Le code source est exécuté ligne par ligne par un interpréteur. Cela signifie que le programme est lu et exécuté en temps réel, sans être préalablement converti en code machine. Exemples : JavaScript est souvent exécuté dans un navigateur web, où l'interpréteur lit et exécute le code directement.

Avantages et inconvénients

- **Langages compilés** : Les langages compilés sont généralement plus rapides à l'exécution, car le code machine est directement exécutable par l'ordinateur. Cependant, ils nécessitent une étape de compilation avant l'exécution, ce qui peut ralentir le processus de développement. Par exemple, le langage C#, utilisé avec Unity pour le développement de jeux, est compilé afin de produire un fichier exécutable qui peut être lancé sur différentes plateformes.
- **Langages interprétés** : Les langages interprétés sont souvent plus flexibles et permettent un développement plus rapide, car ils n'ont pas besoin d'une étape de compilation. Cependant, ils peuvent être plus lents à l'exécution, car chaque ligne de code doit être interprétée en temps réel. Par exemple, JavaScript est interprété par le navigateur, ce qui permet aux développeurs de modifier le code et de voir immédiatement les résultats sans avoir à recompiler le programme.

Pourquoi apprendre la programmation ?

Apprendre la programmation offre de nombreux avantages :

- **Résolution de problèmes** : La programmation développe des compétences en résolution de problèmes et en logique.
- **Créativité** : Elle permet de créer des applications, des jeux, des sites web, et bien plus encore.
- **Carrière** : La demande pour les compétences en programmation est élevée dans de nombreux secteurs, offrant de nombreuses opportunités d'emploi.

- **Automatisation** : Elle permet d'automatiser des tâches répétitives, augmentant ainsi l'efficacité personnelle et professionnelle.
- **Compréhension des technologies** : En apprenant à programmer, on acquiert une meilleure compréhension des technologies qui nous entourent, ce qui est essentiel dans un monde de plus en plus numérique.

Concepts principaux de la programmation

Tous les langages de programmation partagent certains concepts fondamentaux, bien que leur syntaxe et la manière de les écrire puissent varier. Voici quelques concepts clés :

- **Variables** : Des conteneurs pour stocker des données. Elles peuvent contenir différents types de valeurs, comme des nombres, des chaînes de caractères, ou des booléens.
- **Contrôle de flux** : Des structures qui déterminent l'ordre dans lequel les instructions sont exécutées, comme les boucles (`for`, `while`) et les conditions (`if`, `else`).
- **Fonctions** : Des blocs de code réutilisables qui effectuent une tâche spécifique. Elles permettent de structurer le code et de le rendre plus lisible.
- **Structures de données** : Des moyens d'organiser et de stocker des données, comme les tableaux, les listes, et les dictionnaires.
- **Objets** : Dans les langages orientés objet, les objets sont des structures de données qui permettent de modéliser des éléments du monde réel dans le code. Par exemple, un objet `Voiture` pourrait avoir des propriétés (caractéristiques) comme `couleur` et `marque`, et des méthodes (actions) comme `démarrer()` ou `freiner()`.

L'objectif de cette session est de vous familiariser avec ces concepts de base et de vous donner une compréhension générale de ce qu'est la programmation. Au fur et à mesure que vous progresserez, vous apprendrez à utiliser ces concepts pour écrire des programmes simples et résoudre des problèmes concrets.

JavaScript

Pour le cours, nous utiliserons le langage JavaScript, qui est un langage de programmation interprété, principalement utilisé pour le développement web. JavaScript est omniprésent dans les navigateurs web et permet de créer des interfaces interactives et dynamiques. Par exemple, il est utilisé pour valider des formulaires, créer des animations, et gérer des événements utilisateur. En apprenant JavaScript, vous acquerez des compétences précieuses pour le développement web moderne. De plus, nous utiliserons le langage dans un cadre de jeu afin de vous initier à la programmation de manière ludique et engageante et vous préparer à des projets plus complexes où vous pourrez transférer vos connaissances à d'autres langages de programmation (C#, etc.).

Attention : JavaScript et Java sont deux langages de programmation différents, malgré la similarité de leurs noms. Java est un langage compilé, tandis que JavaScript est interprété. Ils ont des syntaxes et des usages différents. Java est souvent utilisé pour le développement d'applications Android et d'applications d'entreprise, tandis que JavaScript est principalement utilisé pour le développement web. Évitez de les confondre 😊.

Qu'est-ce qu'un algorithme?

Un algorithme est une série d'instructions ou d'étapes à suivre pour résoudre un problème ou accomplir une tâche. En programmation, les algorithmes sont essentiels car ils définissent la logique et la séquence des

opérations à effectuer. Par exemple, un algorithme pour trier une liste de nombres pourrait impliquer de comparer chaque nombre avec les autres et de les réorganiser en conséquence.

Cela permet de planifier et de structurer le code avant de l'écrire, facilitant ainsi la compréhension et la maintenance du programme. Les algorithmes peuvent être simples, comme une recette de cuisine, ou complexes, comme ceux utilisés dans les moteurs de recherche ou les systèmes de recommandation.

Les programmes compliqués, comme les jeux vidéo, sont souvent réfléchis à l'avance sous forme d'algorithmes, puis décomposés en parties plus petites et plus gérables. Cela permet de développer plus facilement des fonctionnalités complexes en les construisant à partir de composants plus simples et éviter les erreurs. Par exemple, dans un jeu vidéo, un algorithme pourrait définir comment un personnage se déplace, comment les ennemis réagissent, ou comment les scores sont calculés. En comprenant et en utilisant des algorithmes, les programmeurs peuvent créer des logiciels efficaces et performants qui répondent aux besoins des utilisateurs.

Représentation d'un algorithme

Il existe plusieurs façons de représenter un algorithme, notamment :

- **Pseudocode** : Une description textuelle de l'algorithme, utilisant une syntaxe proche du langage naturel. Le pseudocode est souvent utilisé pour planifier un algorithme avant de le coder dans un langage de programmation spécifique. Le pseudocode permet de se concentrer sur la logique de l'algorithme sans se soucier des détails syntaxiques d'un langage particulier. Il est facile à lire et à comprendre, même pour ceux qui ne sont pas familiers avec la programmation. Il utilise des mots-clés comme "Si", "Alors", "Pour", et "Tant que" pour décrire les conditions et les répétitions. Le pseudocode est un outil précieux pour les programmeurs, car il permet de clarifier la logique d'un algorithme avant de le traduire en code.

Par exemple, un pseudocode pour un algorithme de tri pourrait ressembler à ceci :

```
```\nDébut\n  Pour chaque élément dans la liste\n    Comparer l'élément avec les autres éléments\n    Si l'élément est plus grand, échanger les positions\n  Répéter jusqu'à ce que la liste soit triée\nFin\n```\n
```

Un algorithme pour déplacer un personnage dans un jeu vidéo pourrait être écrit en pseudocode comme suit :

```
Début\n Tant que le jeu est en cours\n Si la touche "haut" est pressée\n Déplacer le personnage vers le haut de 2 pixels\n Si la touche "bas" est pressée\n Déplacer le personnage vers le bas de 2 pixels\n Si la touche "gauche" est pressée\n
```

```
 Déplacer le personnage vers la gauche de 2 pixels
 Si la touche "droite" est pressée
 Déplacer le personnage vers la droite de 2 pixels
 Fin Tant que
Fin
```

- **Organigramme ou diagramme de flux** : Une représentation visuelle de l'algorithme, utilisant des symboles pour illustrer les différentes étapes et décisions dans le processus. Les organigrammes sont utiles pour visualiser la structure d'un algorithme et comprendre comment les différentes parties interagissent.







